Chapter 14 In-Class Exercise: Reading Student Data from a File

Prof. Jordan C. Hanson

October 28, 2026

1 A Simple Course Class

Suppose we want a `Course` object to store a course name and a list of students.

```
class Course:
    def __init__(self, name):
        self.name = name
        self.students = []
    def add_student(self,id,name):
        self.students.append([id, name])
```

Create the course:

```
course = Course("COSC 120")
```

At this point, what is stored in `course.students`?

2 Student Data in a File

Create a text file named

```
students.txt
```

containing

```
2601001,Ana
2601002,Ben
2601003,Chloe
2601004,Diego
2601005,Elena
```

3 Open and Read the File

Python can open a file with

```
with open("students.txt", "r") as f:
```

The `"r"` means **read mode**.
Complete the loop:

```
with open("students.txt", "r") as f:
    for line in f:
        id, name = line.strip().split(",")
        course.add_student(int(id),name)
```

4 Examine the Course Object

After the file has been read, run

```
print(course.name)
print(course.students)
```

How many student records should now be stored? Now print them one at a time:

```
for student in course.students:
    print(student)
```

5 Add a Useful Method

Add this method to `Course`:

```
def number_of_students(self):
    return len(self.students)
```

Then run

```
print(course.number_of_students())
```

What should be printed?

6 Your Turn

Add another line to `students.txt`:

```
2601006,Fatima
```

Run the program again.

1. Does the Python code need to be changed?
2. How many records are now stored?
3. Why is reading data from a file more useful than typing every student directly into the Python program?

Key idea: The file stores the *data*; the `Course` class stores and organizes that data after Python reads it.